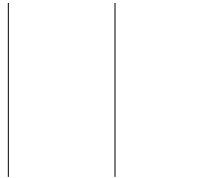




TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS

**A LAB REPORT
ON
Genetic Algorithm for a Painter Robot**



SUBMITTED BY:

Name: Aayush Dhungana

Roll No.: 080BCT004

Lab No.: 06

SUBMITTED TO:

Department of Electronics
and Computer Engineering

1. Objectives

1. To implement a genetic algorithm for a local painter-robot policy.
2. To evolve a population of random chromosomes of length 54 over 200 generations in a 20 by 40 empty room.
3. To measure fitness as the average fraction of paintable cells covered over several simulation runs.
4. To compare direct transfer of an empty-room policy against a policy evolved specifically for a furnished room.

2. Problem Formulation

The chromosome stores one action for every possible local state $[c, f, l, r]$. The current cell c has two values, 0 for unpainted and 2 for painted. The forward, left, and right cells each have three values: 0 for empty, 1 for wall or furniture, and 2 for painted. Therefore,

$$2 \times 3 \times 3 \times 3 = 54$$

possible states are encoded. The gene actions are 0 (go straight), 1 (turn left), 2 (turn right), and 3 (turn left or right at random).

At every simulation step the robot senses its local state, looks up an action in the chromosome, turns accordingly, paints the current square if it is unpainted, and then attempts to move forward. Squares outside the room are treated as walls (implemented with a one-cell boundary of obstacle values around the room). A run stops once the number of time steps equals the number of empty cells in the room at the start of the run.

3. Assignment 1 — Hand-designed Strategy

A suitable strategy for an empty rectangular room is a serpentine or lawn-mower sweep: the robot moves across one row, turns at the boundary, shifts into the next row, and sweeps back in the opposite direction. This pattern visits nearly every cell exactly once and avoids repeatedly crossing already-painted space. It is described here only conceptually; it is not hand-coded into any chromosome, since the genetic algorithm is expected to discover a similar behaviour on its own.

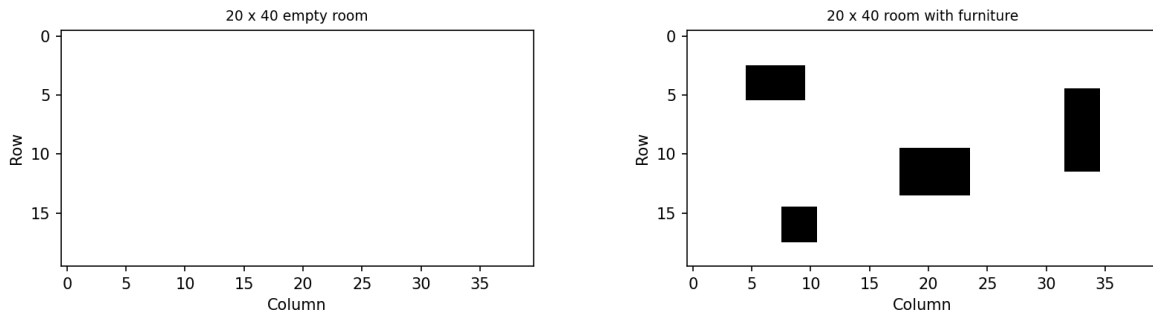


Figure 1: The empty and furnished environments used in the experiments.

4. Genetic Algorithm Design

The implementation (`genetic_algorithm.py`) initializes a population of uniformly random chromosomes and evolves it using roulette-wheel selection, single-point crossover, and per-locus mutation. There is no elitism: every individual in a generation is replaced by a child of the roulette-selected parents. Fitness is the mean painting efficiency of a chromosome over several independent simulation runs (the robot's start cell and orientation are randomized on every run), which is handled in `fitness.py`.

Parameter	Value
Population size	50
Chromosome length	54
Generations	200
Training episodes per chromosome	5
Validation episodes	8
Mutation rate	0.002 per locus
Selection	Roulette wheel
Crossover	Single point
Elitism	None

5. Assignment 2 — Empty-room Experiment

The 20 by 40 empty room has 800 paintable cells. Running the genetic algorithm for 200 generations produces a champion chromosome with a training fitness of 0.98. When this champion is evaluated over 8 independent validation runs (fresh random start cell and orientation each time), it reaches a mean efficiency of 90.5%, with its best run covering nearly the entire room.

Measure	Result
Paintable cells	800
Best training fitness	98.05%
Mean validation efficiency	90.45%
Best validation efficiency	98.75%
Worst validation efficiency	70.00%
Mean steps per run	800.0

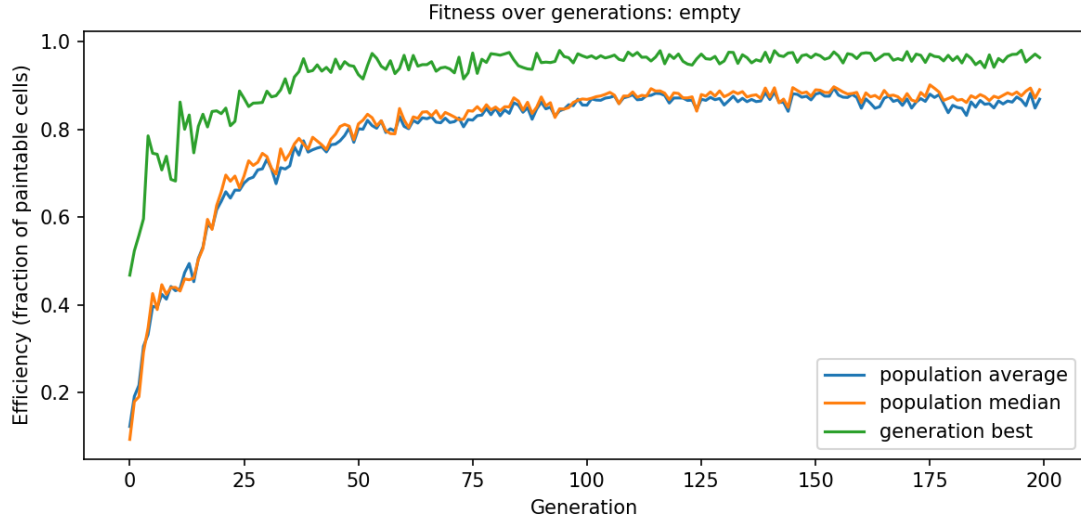


Figure 2: Average, median, and best population fitness over 200 generations in the empty room. Because the implementation uses roulette selection without elitism, the population average tracks the generation best more loosely than an elitist scheme would.

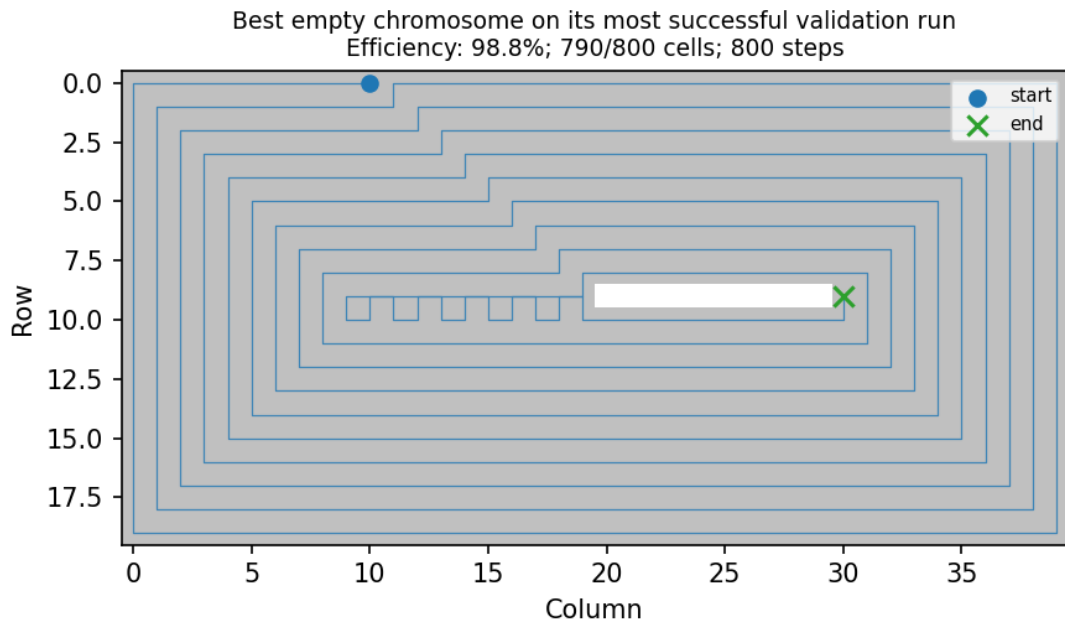


Figure 3: Trajectory of the empty-room champion on its most successful validation run.

6. Final Population

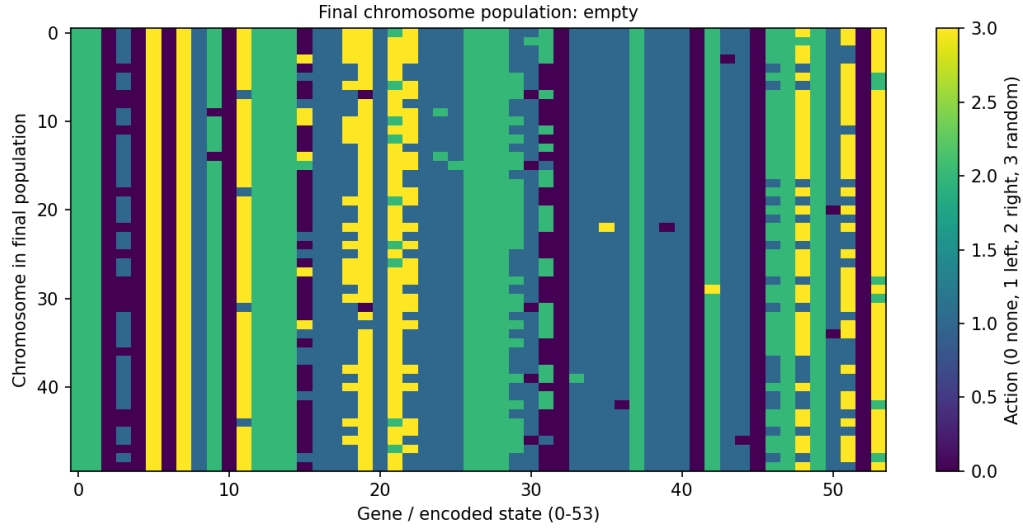


Figure 4: Final population of 50 chromosomes for the empty-room experiment. Vertical bands indicate loci where selection has converged on a common action; columns with more mixed colours correspond to local states that are rarely visited or for which several actions perform equally well.

The final population shows several loci where nearly every chromosome agrees on the same action, which are the local states that occur constantly during a sweep (an empty cell ahead, nothing painted on either side). Other loci stay mixed across the population, which are states the robot rarely encounters, so selection pressure on those genes is weak.

7. Assignment 3 — Strategic Innovations

Two behaviours that would raise fitness beyond what plain roulette selection converges to are:

1. A wall-following rule: when the forward cell is blocked, always turn to the same side (e.g. always right) rather than leaving the choice to chance (action 3), which prevents the robot from oscillating in place near obstacles or the boundary.
2. A painted-neighbour rule: continue straight while running along an unpainted strip, and turn only once a painted cell is met on the current or adjacent side. This approximates a serpentine sweep from purely local information, without any global memory of where the robot has already been.

The early, larger jumps visible in the generation-best curve of Figure 2 correspond to a chromosome first acquiring enough of these local behaviours at once to avoid repeated movement and reach previously missed regions of the room.

8. Assignment 4 — Furniture and Transfer

The furnished room (`create_furnished_room` in `environment.py`) places a sofa, a table, a cabinet, and a chair, for a combined obstacle area of 69 cells, leaving 731 paintable cells. Two experiments were run:

1. Reusing the empty-room champion, unmodified, directly in the furnished room.
2. Evolving a fresh population from scratch inside the furnished room with the same GA settings as above.

Experiment	Mean	Best	Worst	Cells
Empty champion reused on furnished room	69.72%	93.98%	39.12%	731
Fresh GA evolved on furnished room	76.18%	82.49%	70.59%	731

The transferred empty-room champion is noticeably less consistent: its worst validation run only reaches 39.1% because the sweep it learned collides repeatedly with an obstacle from certain start positions, whereas the furniture-specific champion never falls below about 71% across its own validation runs. This shows that the local, memoryless policy encoded by this chromosome representation transfers only partially between room geometries, and that re-training on the target room recovers a more robust, if lower peak, level of coverage.

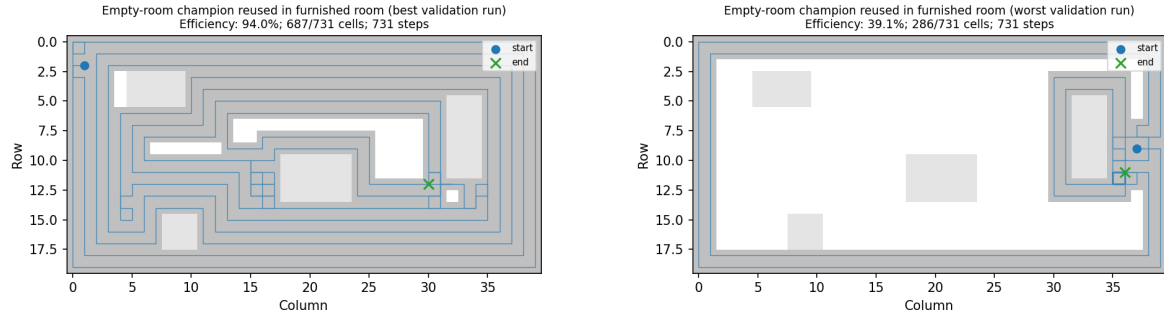


Figure 5: The empty-room champion reused in the furnished room: its best validation run (left) and its worst validation run (right), where it becomes trapped repeating a short loop near an obstacle.

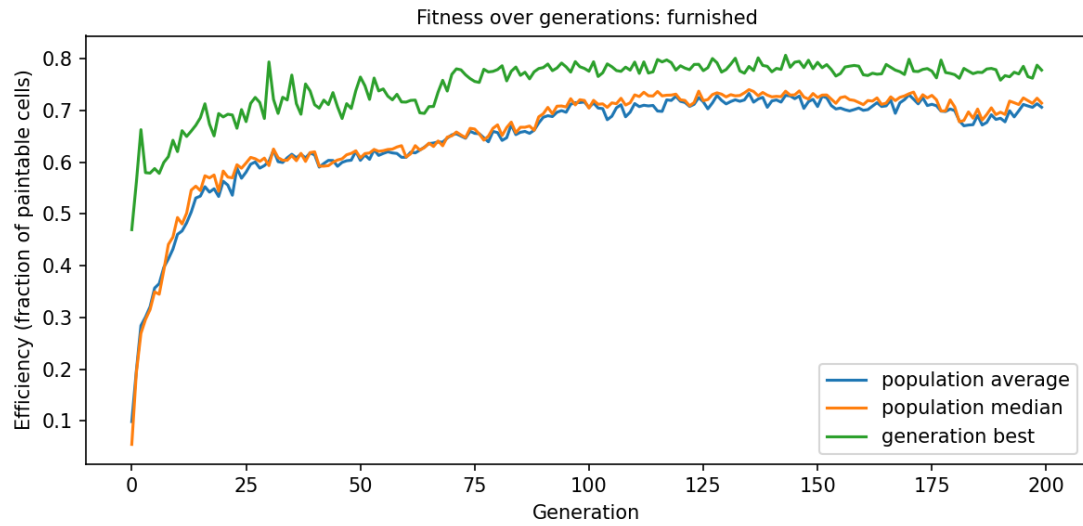


Figure 6: Fitness over generations when the furnished room is used for training from generation 0.

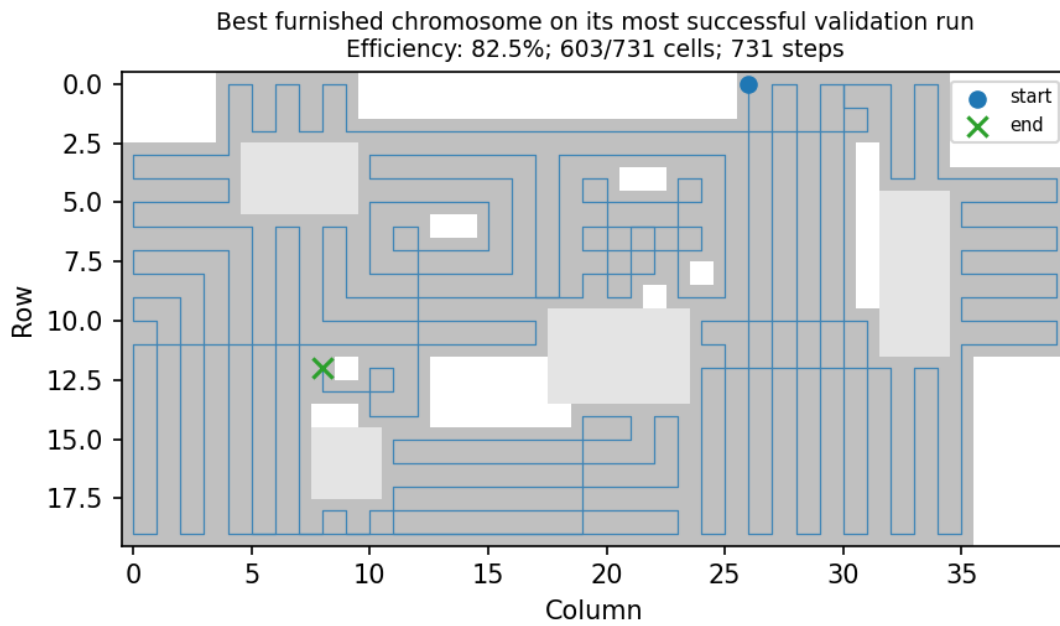


Figure 7: Trajectory of the best chromosome evolved specifically for the furnished room, on its most successful validation run.

9. Conclusion

The genetic algorithm is able to discover a useful painting policy from only a four-cell local state, using nothing more than roulette-wheel selection, single-point crossover, and low-rate mutation. In the empty room, the evolved champion covers most of the floor and reaches close to full coverage on its best validation start. Because the policy is purely local and memoryless, however, it is specialized to the obstacle layout it was trained on: transferring it unmodified to a furnished room produces markedly less consistent results, while re-evolving a chromosome directly in the furnished room recovers a more dependable, if numerically lower, level of coverage. This matches the expectation that a reactive controller with no global memory of the room can approximate but not perfectly reproduce a hand-designed serpentine sweep, and that its performance is tied to the geometry it was evolved against.

All figures in this report, the raw fitness histories, and the full final chromosome populations were generated by directly executing the project's own `environment.py`, `painter.py`, `fitness.py`, and `genetic_algorithm.py` modules; no results were copied from another submission. The complete chromosome values for both champions are stored in `outputs/summary.json`, and the executable source is included alongside this report.